

Introduction to Python

Jupyter Notebooks, Variables, and Data Types

Victor Wandera Lumumba, MSc.
Statistician — Data Analyst — ML Expert

Mediacrest Training College

June 17, 2026

Session Objectives

- Understand the role of Python in Data Analysis and AI.
- Navigate and utilize Jupyter Notebooks effectively.
- Define variables and follow Python naming conventions.
- Identify and manipulate core Python data types (Numeric, Strings, Booleans).
- Get a preview of essential data structures and libraries.

Why Python for Data Science?

- **Simplicity & Readability:** Clean syntax allows focus on problem-solving, not language quirks.
- **Rich Ecosystem:** Thousands of specialized libraries (Pandas, NumPy, Scikit-Learn).
- **Community Support:** Massive global community, extensive documentation, and forums.
- **Versatility:** Seamlessly handles everything from data wrangling to deep learning and web deployment.

The Python Ecosystem

- **Core Language:** The foundation (variables, loops, functions).
- **Data Manipulation:** Pandas, NumPy.
- **Visualization:** Matplotlib, Seaborn, Plotly.
- **Machine Learning:** Scikit-Learn, TensorFlow, PyTorch.
- **Environment:** Jupyter, VS Code, PyCharm.

What is Jupyter Notebook?

Definition

An open-source web application that allows you to create and share documents containing live code, equations, visualizations, and narrative text.

- Originally "IPython Notebook", now part of Project Jupyter.
- Supports over 40 programming languages, including Python, R, and Julia.
- The industry standard for interactive data exploration.

Why Jupyter for Data Analysis?

- **Interactive Exploration:** Run code in chunks and see results immediately.
- **Storytelling:** Combine code, output (charts/tables), and Markdown explanations.
- **Reproducibility:** Share the entire analytical workflow in a single file (`.ipynb`).
- **Prototyping:** Ideal for testing hypotheses before building production pipelines.

Navigating the Jupyter Interface

- **Dashboard:** File browser, running terminals, and notebooks.
- **Notebook App:** The main document interface.
- **Menu Bar:** File, Edit, View, Insert, Cell, Kernel, Help.
- **Toolbar:** Quick access to save, add cells, cut/copy/paste, run, stop.

Understanding Jupyter Cells

- **Code Cells:** Where you write and execute Python code.
 - Output is displayed directly below the cell.
- **Markdown Cells:** For formatted text, headings, links, and LaTeX equations.
 - Used to document your analysis and tell a data story.

Executing Code & Shortcuts

- **Run Cell:** Shift + Enter (Run and select below)
- **Run & Insert:** Alt + Enter (Run and insert below)
- **Run & Stay:** Ctrl + Enter (Run and stay in current cell)
- **Interrupt Kernel:** Stop long-running code (Esc + I + I or the stop button).
- **Command vs. Edit Mode:** Press Esc for Command mode, Enter for Edit mode.

Python Basics: Variables & Data Types

What is a Variable?

Definition

A variable is a reserved memory location to store values. It acts as a container or a "label" for data.

- In Python, you don't need to declare a variable's type explicitly.

```
1 age = 25
2 name = "Victor"
```

Variable Naming Rules

- Must start with a letter (a-z, A-Z) or an underscore (_).
- Cannot start with a number.
- Can contain letters, numbers, and underscores.
- Case-sensitive (age, Age, and AGE are different).
- Cannot be a Python reserved keyword (e.g., `class`, `for`, `if`).
- **Best Practice:** Use descriptive, lowercase names with underscores (snake_case).

Python's Dynamic Typing

- Python is **dynamically typed**.
- The interpreter infers the data type at runtime based on the assigned value.
- A variable can change its type during execution.

```
1 x = 10          # x is an integer
2 x = "Hello"     # x is now a string
```

Note

Flexibility is great, but requires careful tracking of data types to avoid runtime errors!

Core Python Data Types

- **Numeric Types:** `int` (integers), `float` (decimals), `complex`.
- **Text Type:** `str` (strings).
- **Boolean Type:** `bool` (True or False).
- **Sequence Types:** `list`, `tuple`, `range`.
- **Mapping Type:** `dict` (dictionary).
- **Set Types:** `set`, `frozenset`.

Today's Focus: Numeric, String, and Boolean types.

Numeric Types: Integers & Floats

- **Integer (int):** Whole numbers, positive or negative, without decimals.
- **Float (float):** Real numbers with a decimal point.

```
1 students = 45
2 temperature = -5
3
4 pi = 3.14159
5 gpa = 3.85
```

Basic Arithmetic Operations

- Addition (+), Subtraction (-), Multiplication (*), Division (/).
- **Floor Division (//)**: Divides and rounds down to the nearest whole number.
- **Modulus (%)**: Returns the remainder after division.
- **Exponentiation (**)**: Raises a number to a power.

```
1 print(10 / 3)      # 3.3333...
2 print(10 // 3)     # 3
3 print(10 % 3)      # 1
4 print(2 ** 3)      # 8
```

Strings (Text Data)

- **String (str):** A sequence of characters enclosed in single (' ') or double (" ") quotes.

```
1 course = "Data Analysis"  
2 initial = 'V'
```

- **Triple Quotes:** Used for multi-line strings or docstrings.

```
1 description = """This is a  
2 multi-line string."""
```

String Manipulation & Methods

- **Concatenation:** Join strings using +.
- **Repetition:** Repeat strings using *.
- **Common Methods:**
 - .upper(), .lower()
 - .strip()
 - .replace(old, new)
 - .split(delimiter)

```
1 text = "  Hello World  "  
2 print(text.strip().lower())  
3 # Output: hello world
```

String Indexing and Slicing

- Strings are **ordered sequences**.
- **Indexing:** Access a character using `[index]`.
- Python uses **zero-based indexing**.
- **Slicing:** Extract a substring using `[start:stop:step]`.

```
1 word = "Python"
2
3 print(word[0])      # 'P'
4 print(word[-1])     # 'n'
5 print(word[0:3])    # 'Pyt'
6 print(word[::-1])   # 'nohtyP'
```

Booleans Logical Operators

- **Boolean (bool):** Represents one of two values: True or False.
- Used for logic, filtering data, and conditional statements.
- **Comparison Operators:** ==, !=, >, <, >=, <=.
- **Logical Operators:** and, or, not.

```
1 is_enrolled = True
2 age = 20
3
4 check = (age >= 18) and is_enrolled
5 print(check) # True
```

Type Conversion (Casting)

- Converting one data type to another explicitly.
- Common functions: `int()`, `float()`, `str()`, and `bool()`.

```
1 x = "10"
2 y = int(x) + 5
3
4 z = 3.14
5 s = str(z)
6
7 print(y)          # 15
8 print(s)          # "3.14"
9
10 print(bool(0))    # False
11 print(bool(42))   # True
```

Checking Data Types

- Use the built-in `type()` function to determine the type of a variable.
- Very useful for debugging and data analysis.

```
1 revenue = 15000.50
2 print(type(revenue))
3
4 is_active = True
5 print(type(is_active))
```

Basic Data Structures

Lists (Ordered Collections)

- **List (list):** A mutable, ordered collection of items.
- Lists can store mixed data types.
- Created using square brackets [].

```
1 scores = [85, 90, 78, 92]
2 mixed = [1, "Text", True, 3.14]
3
4 scores.append(88)
5
6 print(scores)
7 print(scores[0])    # 85
```

Dictionaries (Key-Value Pairs)

- **Dictionary (dict):** Stores data as key-value pairs.
- Optimized for fast lookup by key.
- Created using curly braces `{}`.

```
1 student = {  
2     "name": "Alice",  
3     "age": 22,  
4     "course": "Data Science"  
5 }  
6  
7 print(student["name"])    # Alice
```

Why Data Structures Matter

- **Data Organization:** Real-world data is rarely a single number or string.
- **Efficiency:** Structures like dictionaries allow fast lookups and mapping.
- **Foundation for DataFrames:** A Pandas DataFrame is essentially a collection of Series.
- Mastering lists and dictionaries is critical for JSON parsing and API data wrangling.

Preview: The Data Science Stack

- Once you master core Python, you unlock the “superpowers”:
- **NumPy**: Fast numerical computing and N-dimensional arrays.
- **Pandas**: Data manipulation and analysis using DataFrames.
- **Matplotlib/Seaborn**: Creating data visualizations.
- **Scikit-Learn**: Machine learning algorithms and tools.

Introduction to NumPy

- **Numerical Python.**
- Core object: The `ndarray` (N-dimensional array).
- Faster and more memory-efficient than standard Python lists for numerical operations.
- **Use cases:** Linear algebra, statistics, and large numerical datasets.

Introduction to Pandas

- **Python Data Analysis Library.**
- Core objects: Series (1D) and DataFrame (2D tabular data).
- Supports reading and writing data from CSV, Excel, and SQL sources.
- Handles missing data, merging, joining, and time-series operations.
- *We will dive deeper into Pandas in the next module!*

Best Practices in Python Coding

- **Readability Counts:** Follow PEP 8 guidelines.
- **Meaningful Names:** Use `total_revenue` instead of `tr`.
- **Comment Your Code:** Explain *why*, not only *what*.
- **Modularize:** Break complex tasks into functions.
- **Version Control:** Use Git/GitHub for tracking changes and collaboration.

Common Beginner Pitfalls

- **Indentation Errors:** Python relies on whitespace. Mixing tabs and spaces causes `IndentationError`.
- **Off-by-One Errors:** Forgetting that indexing starts at 0.
- **Type Mismatches:** Adding a string and an integer without conversion.
- **Mutable Defaults:** Avoid using mutable objects as default function arguments.

Hands-on Practice

- ➊ **Exercise 1:** Create variables for a dataset (e.g., patient age, diagnosis, and is_treated).
- ➋ **Exercise 2:** Calculate the average age from a list of ages.
- ➌ **Exercise 3:** Create a dictionary representing a student record and extract specific fields.
- ➍ **Exercise 4:** Format a string to output: "Patient [Name] is [Age] years old."

Action

Open your Jupyter Notebooks and let's code!

Summary & Key Takeaways

- Python is widely used for Data Analysis because of its simplicity and ecosystem.
- Jupyter Notebooks provide an interactive and reproducible environment.
- Variables store data, and Python uses dynamic typing.
- Understanding data types and structures is fundamental.
- Next up: Data manipulation with Pandas!

Questions?

Victor Wandera Lumumba

Statistician — Data Analyst — ML Expert

Email: lumumbavictor172@gmail.com

Web: beyonddataanalytics.online

GitHub: [Lumumba1992](https://github.com/Lumumba1992)